

# Signal Desk

A working problem-sensing and response pipeline: scan recurring AI-tooling complaints, rank them into an opportunity matrix, then draft (never auto-send) outreach to the people describing the pain.

**Team** · Divya, Noa, Sreya, Harsh

**Course** · AI Tools for Venture Workflows, Session 1

**Prepared by** Div

# How we picked terms and platforms

We did not start with a venture idea. We let the scan surface one, so the picking process itself needed to be disciplined.

## Search lexicon

Broad emotion terms alone returned marketing copy, not complaints. We layered three word classes together.

product names   Cursor   Claude Code   Copilot  
Operator   Comet   + symptom words   loses context  
hallucinate   repeats itself   + venue filter  
site:reddit.com   site:news.ycombinator.com

## Platforms scanned

- Reddit (search index + direct fetch attempts)
- Hacker News (search index + item pages)
- GitHub Issues on public agent repos
- G2 and review aggregators
- Dev.to and technical Substacks
- X / Twitter · attempted, no free access (see slide 09)

# Problem-opportunity matrix

Ranked by a combination of how often the complaint recurs, how much damage it does, and whether we could actually find and message a concentrated group of people describing it.

| RANK | PROBLEM CLUSTER                                                         | FREQUENCY | SEVERITY | CONCENTRATED IN                                 | OPPORTUNITY READ                                                                                  |
|------|-------------------------------------------------------------------------|-----------|----------|-------------------------------------------------|---------------------------------------------------------------------------------------------------|
| 01   | Coding agents lose context / repeat finished work in long sessions      | High      | High     | GitHub issues, r/ClaudeAI, r/cursor, HN, dev.to | Sharp, technical, and people are already hand-building workarounds. Chosen for the live workflow. |
| 02   | "AI agents don't survive production," enterprise reliability skepticism | High      | Medium   | HN essays, thought-leadership blogs             | Loud but diffuse. Hard to turn an essay reader into a discovery call.                             |
| 03   | AI support agents can't escalate or retain ticket context               | High      | High     | G2 reviews, vendor comparison sites             | Real pain, but an enterprise sales motion already crowded with incumbents                         |
| 04   | AI browser / computer-use agents too slow for daily use                 | Medium    | Medium   | Newsletter and Medium first-person reviews      | Pre-chasm category, no clear repeatable wedge yet                                                 |
| 05   | AI notetakers raise privacy and legal exposure                          | Medium    | Medium   | Legal and HR trade press                        | Policy-adjacent, slower enterprise sales cycle                                                    |

# Context loss in long agentic coding sessions

The people describing this are specific, findable, and already paying for the tools that are frustrating them, which makes them reachable for customer discovery, not just a survey population.

## What we actually found

- anthropics/claude-code #92951, "Agent loses context and makes repeated mistakes during extended sessions"
- Dev.to: "Why Your Claude Code Sessions Keep Losing Context (And How to Fix It)"
- Dev.to: "Why Does Claude Code Lose Track Mid-Task?"
- Substack: a user reverse-engineering why /compact discards useful context

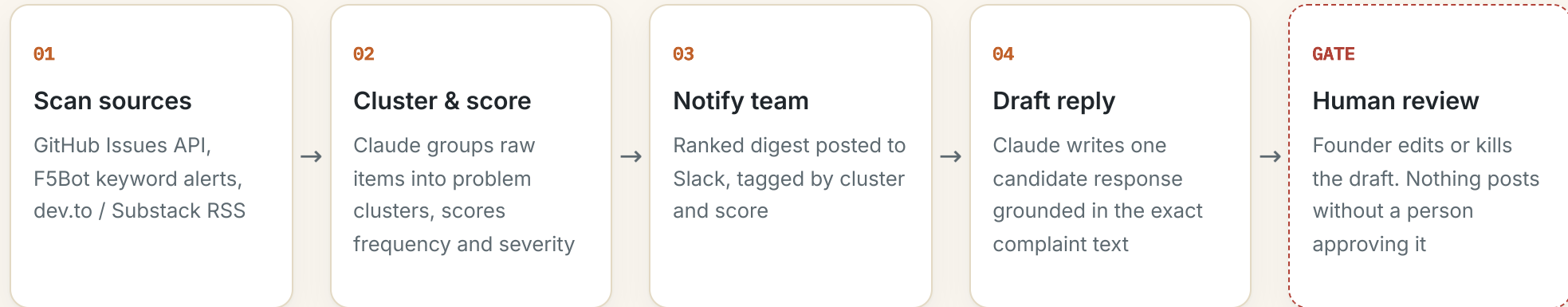
## Who is most concentrated here

Developers running multi-day or multi-hundred-tool-call agent sessions on Claude Code, Cursor, Windsurf, and Copilot, most visible in GitHub issue threads on those repos and in r/ClaudeAI and r/cursor.

The tell that this is a real wedge, not just noise: people are already hand-rolling fixes (checkpoint files, manual spec re-statements) instead of waiting for the vendor.

# Scan, cluster, alert, draft, then a human decides

Everything up to the send button is automatable. The send button is not.



# What we picked at each stage, and why

| STAGE                   | TOOL              | WHY THIS ONE                                                                                            | COST      |
|-------------------------|-------------------|---------------------------------------------------------------------------------------------------------|-----------|
| Scan (technical)        | GitHub Issues API | Direct source of acute, reproducible complaints with technical detail attached                          | Free      |
| Scan (social, tripwire) | F5Bot             | Simplest free keyword alert across Reddit and HN, good enough as an early warning                       | Free      |
| Scan (upgrade path)     | Syften            | Broader coverage across Reddit, HN, Discord, Slack, and X once free tripwires prove the terms are right | Paid      |
| Cluster & score         | Claude            | Reads noisy, inconsistent text and groups it by root cause faster than a human triage pass              | Free tier |
| Notify                  | Slack webhook     | Lands where the team already works, no new habit to build                                               | Free      |
| Draft reply             | Claude            | First-pass reply grounded in the actual complaint, cheap enough to regenerate per item                  | Free tier |

# The two prompts doing the real work

## PROMPT A: CLUSTER AND SCORE

You are a research analyst screening startup problem signal.  
For each raw item I give you (from GitHub issues, forums, or reviews):

1. Assign it to a problem cluster. Name the cluster as a specific failure in a job-to-be-done, not a vague category.
2. Score FREQUENCY 1-5, how often this same complaint recurs.
3. Score SEVERITY 1-5, how much workflow, financial, or trust damage the complaint describes.
4. Note WHERE CONCENTRATED, platform, subreddit, or repo.
5. Flag it if the poster describes a workaround they built themselves. That is a stronger signal than the complaint alone.

Return one row per cluster, aggregating duplicates, as a table.

Raw items: [paste scraped items]

## PROMPT B: DRAFT A REPLY (ASTROTURF GUARDRAILS BUILT IN)

Draft a reply from [founder name], building tooling for [cluster].  
Here is the exact complaint and source link: [paste complaint + link]

Rules:

- Under 80 words. Sign with the founder's real name.
- Reference the specific detail they described, not a summary.
- Offer something useful first (a workaround, a direct answer) before making any ask.
- If asking for time, frame it as customer discovery, never as a pitch or a beta invite.
- Do not claim experience we do not have.
- If the exchange goes past one reply, disclose we are building something in this space.
- No marketing language.

# Two drafts the workflow produced

Both are illustrative composites built from the real patterns in slide 03, not messages we sent, and not tied to one identifiable poster.

Pattern seen in a GitHub issue thread · agent lost context around 40 tool calls, redid finished work

Hit this too, right around the same tool-call count. Feels more like a compaction problem than a model one. One thing that helped me was forcing a written checkpoint file every N steps instead of trusting the summary. If you're open to it, I'd love 15 minutes to hear what your session looked like right before it broke. Not selling anything, just trying to map where this fails.

Div.

Pattern seen in a dev.to comment thread · re-explained the same architecture decision three times in one session

Three times in one session is rough. Was that right after a compact, or just long natural drift? A few of us are mapping this exact pattern, agents forgetting decisions that were already settled. Would fifteen minutes of your actual experience be useful to trade for whatever we learn along the way? No pitch attached.

Div.

# Fast and easy versus astroturfing

The same automation that makes outreach cheap for a founder is what makes it read as manufactured if we are not careful. Here is where we drew the line.

- 01 Every reply discloses a real name and, past one exchange, that we are building something in this space. No sock accounts, no "just a fellow user" framing.
- 02 The draft always offers something before it asks for anything, a workaround, a direct answer, a resource, never an ask-first message.
- 03 Automation stops at the draft. A person reads, edits, and approves every message before it posts or sends. Nothing goes out unattended.
- 04 Volume is capped intentionally. A founder who replies to forty threads a day in identical cadence reads as a bot even when a human wrote every word.
- 05 Every sent message is logged, so the team can check for repetition or drift toward generic, scripted-sounding language over time.

# Real limitations we hit building this

## Direct scraping got blocked

Fetching Reddit's own search pages directly was blocked outright. Hacker News item pages rate-limited us on a second request. Confirms the assignment's own tool list: you need F5Bot, Syften, or an official API, not ad hoc fetching.

## Broad queries returned marketing, not complaints

"AI agent frustrating" mostly surfaced explainer blogs and thought-leadership about agents. Real complaint density only showed up once we narrowed to a specific product name plus a specific symptom word plus a specific venue like GitHub issues or dev.to.

## X was a dead end on the free tier

No free path to X search turned up real posts. The sample tool list treats X API/MCP as a given, but it is a real gap unless someone on the team already has paid access.

## Aggregator sites validate a pattern, not a lead

G2 and review-aggregator content confirmed the support-agent cluster is real at category scale, but gave us nothing individually addressable to actually message. Good for sizing a problem, useless for sourcing outreach targets.

## The alert feed, as the founder would see it

Three simulated alerts. Click "Draft reply" to see Prompt B's output, then approve or discard it, the way a human review step actually works.

GitHub Issues · agent repo · severity 5/5

Awaiting draft

"Agent completely lost context after roughly 40 tool calls and started redoing work it had already finished. Wasted almost an hour of runtime before I noticed."

Hide draft

PROMPT B OUTPUT

Hit this too, right around the same tool-call count. Feels more like a compaction problem than a model one. One thing that helped me was forcing a written checkpoint file every N steps instead of trusting the summary. If you're open to it, I'd love 15 minutes to hear what your session looked like right before it broke. Not selling anything, just trying to map where this fails.

Div.

Approve to sendDiscard

x/ClaudeAI · keyword alert · severity 4/5

Awaiting draft

"Spent my whole morning re-explaining the same architecture decision three times in one session. At what point do I just write it all in a doc and paste it every time?"

Hide draft

PROMPT B OUTPUT

Three times in one session is rough. Was that right after a compact, or just long natural drift? A few of us are mapping this exact pattern, agents forgetting decisions that were already settled. Would fifteen minutes of your actual experience be useful to trade for whatever we learn along the way? No pitch attached.

Div.

Approve to sendDiscard

dev.to comment · keyword alert · severity 3/5

Awaiting draft

"Built myself a scrappy checkpoint file that I paste back in every 20 minutes. Works, but it is one more thing to remember to do."

Hide draft

PROMPT B OUTPUT

That checkpoint habit is basically the workaround we keep seeing, which is what got us paying attention to this. Curious how you decided on 20 minutes specifically, trial and error or did something actually break at that mark? If you're up for a quick call I'd rather hear it straight from you than guess.

Div.

Approve to sendDiscard

# What we would build next

- Wire the GitHub Issues API and an F5Bot digest into an actual Slack channel instead of a mocked feed
- Tighten Prompt B against panel feedback in class, specifically on the disclosure timing rule
- Run 10 real drafted replies past a non-founder teammate as an astroturfing smell test before sending any
- Decide, after real discovery conversations, whether this cluster is a venture or just a good feature for someone else's product

## SOURCES

[GitHub · anthropics/claude-code issue #92951](#)

[Twig · Most Common Complaints About AI Customer Support Tools](#)

[G2 · AI Agents for Customer Support, 7,900+ reviews](#)

[Nate's Newsletter · I threw my real workday at two AI agents](#)

[Hacker News · Most "AI agents" don't survive production](#)